

# Parameter Class-Based Bounds — Design Spec

**Version:** v0.2 (draft for review) **Date:** 2026-08-05 **Status:** Design — review before implementation

---

## 1. Problem

The current ParamTable (FC) and PARAM\_LIMITS (GCS) hold per-param (min, max) bounds that are tuned to individual airframes. There is no single set of per-param limits that suits all airframes:

- Headroom for the most demanding airframe makes sliders coarse for small-value frames.
- Limits end up hand-bumped per frame (e.g. the 12-tag edit in August 2026) and drift out of sync between params.c and parameters.py.

Additionally the FC currently **clamps nothing** — tag 17 (ProcessParamsWrite, telem.c:1220-1248) stores the incoming value verbatim, including out-of-range enums ((uint8)u.f truncates but does not range-check). A bogus value (corrupt .af, bad slider interaction, manual serial write) can enter the FC and reach the control loops in flight.

## 2. Objectives

1. **Safety:** the FC bounds every write (enums clamped to their range, floats clamped to a generous sanity ceiling).
2. **Per-airframe precision:** the tuning-slider range comes from the .af file, not from a global table.
3. **Class-based ceilings:** a small ParamClass table (**raw** units) sets the overarching FC hard bound for each gain/angle/rate family. One-off params (enums, distances, floored values, divergent-magnitude gains) keep explicit per-param bounds.
4. **Preserve legacy display:** the ParamTable.scale field is **essential and untouched**. It is GCS-only metadata: display = raw / scale for legacy units, display = raw × DISPLAY\_MULT otherwise. The FC never reads it.
5. **Eliminate the sync ritual:** GCS PARAM\_LIMITS is *derived* from the class table, not hand-mirrored.

## 3. Three-tier bounds model

| Tier | Owner             | Content                                                                              | Role                                                  |
|------|-------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------|
| 1    | FC ParamTable     | per-param raw (lo, hi) — class-derived for gain/unit families, explicit for one-offs | <b>Hard clamp</b> on every write; generous by design  |
| 2    | .af file [LIMITS] | per-airframe per-param raw (lo, hi) — optional                                       | Precise <b>tuning-slider range</b> in GCS             |
| 3    | GCS PARAM_LIMITS  | derived from class table                                                             | <b>Fallback</b> slider range when .af has no [LIMITS] |

### 3.1 Units and the role of scale

The FC works entirely in **raw** units — it never reads `ParamTable.scale` (verified: no `.scale` access at runtime in the FC). `scale` is consumed only by the GCS, and only to convert between raw and display values:

```
FC : ParamTable min/max = RAW bounds    (class-derived or explicit)
FC : clamp writes against the raw bounds
GCS : display = raw / scale              (legacy units, legacy checkbox)
      display = raw × DISPLAY_MULT       (everything else; DISPLAY_MULT ≡ 1/scale)
GCS : slider range = raw_bound × display_mult
```

The class table therefore lives in **raw** units — the FC needs no scale math. The GCS converts raw → display exactly as it does today; nothing about the display path changes.

## 4. FC ParamClass design

### 4.1 Class table (RAW-unit ceilings)

Class ceilings are deliberately **generous** — they exist to reject bogus values, not to encode tuned ranges. A class is only used where its members are of comparable magnitude; divergent members become one-offs (§4.3). Values below are **proposals** to be validated against every `.af` file during implementation (there is an existing tuner check that asserts tuned values ≤ limits). All bounds are **raw FC units**.

|                                |                |                               |
|--------------------------------|----------------|-------------------------------|
| <code>classGainRateP</code>    | 0 – 3.0        | (rate P gains; raw)           |
| <code>classGainRateD</code>    | 0 – 0.2        | (rate D gains; raw)           |
| <code>classGainRateI</code>    | 0 – 0.2        | (yaw rate I)                  |
| <code>classGainAngleQ</code>   | 0 – 20         | (quaternion angle P)          |
| <code>classGainAngleI</code>   | 0 – 25         | (angle integral)              |
| <code>classGainAlt</code>      | 0 – 3.66       | (alt position/vel/ROC loop)   |
| <code>classGainNav</code>      | 0 – 15         | (nav position/vel/crosstrack) |
| <code>classGainCam</code>      | 0 – 20         | (camera gimbal)               |
| <code>classGainKf</code>       | 0 – 20         | (KF variances)                |
| <code>classRateIntLim</code>   | 0 – 30         | (rad/s integral clamps)       |
| <code>classAngle</code>        | 0 – 1.047      | (rad – 60°)                   |
| <code>classAngleBipolar</code> | -0.349 – 0.349 | (rad – ±20°)                  |
| <code>classRate</code>         | 0 – 10.472     | (rad/s – 600°/s)              |
| <code>classPct</code>          | 0 – 1.0        | (fraction – 100%)             |
| <code>classPctS</code>         | 0 – 0.5        | (fraction/s – 50%/s)          |
| <code>classHz</code>           | 0 – 255        | (Hz)                          |
| <code>classVolts</code>        | 0 – 255        | (V)                           |
| <code>classAmps</code>         | 0 – 255        | (A)                           |
| <code>classMetres</code>       | 0 – 1000       | (m – fallback only; see 5.2)  |
| <code>classMs</code>           | 0 – 50         | (m/s)                         |
| <code>classMah</code>          | 1500 – 10000   | (mAh)                         |
| <code>classSeconds</code>      | 0 – 60         | (s)                           |

### 4.2 ParamMetaEntry change

Add a `uint8 cls` field (index into `ParamClassBounds[]`). The per-param raw min/max are **computed** from the class:

```
real32 lo = ParamClass[e->cls].lo;    // raw, no scale math
real32 hi = ParamClass[e->cls].hi;    // raw, no scale math
```

scale stays in the entry **unchanged** — it is GCS-only metadata and must never be folded into the bound.

### 4.3 One-offs (classExplicit)

For params that do not fit a shared class — enums, bitmasks, floored values, distances, times, divergent-magnitude gains — the entry carries **explicit** (lo, hi) in the macro call and cls = classExplicit (raw bound = the explicit pair). These are the majority of the table; the class table above covers only the gain/angle/rate families where sharing is meaningful.

### 4.4 Clamp every write

In ProcessParamsWrite (telem.c) and in PackParametersToConfig() (params.c:255), clamp before storing:

```
if (e->type == PARAM_FLOAT)
    u.f = fclamp(u.f, lo, hi);
else
    u.f = (real32) uclamp((uint8)u.f, (uint8)lo, (uint8)hi); // enum range clamp
```

This closes the current gap where out-of-range enums are stored verbatim.

## 5. GCS design

### 5.1 Derived PARAM\_LIMITS

parameters.py gains the ParamClassBounds table (~25 rows, **raw** units — identical numbers to the FC). At import:

```
PARAM_LIMITS[idx] = (cls_lo, cls_hi) # raw — no scale multiply
```

PARAM\_SCALES remains untouched (GCS-only, display conversion). The 128 literal tuples and the AGENTS.md mirror ritual are removed. A unit test recomputes PARAM\_LIMITS from the table and asserts known raw values.

### 5.2 .af per-airframe [LIMITS]

Optional block; values in **raw FC units** (consistent with the rest of the .af file — .af stores FC-native values, see airframes.py \_parse\_value):

```
[LIMITS]
ROLL_RATE_KP = 0, 0.913
ROLL_ANGLE_Q_KP = 0, 11.2
ALT_POS_KP = 0.35, 1.83
```

- Slider range for a param = .af [LIMITS] entry if present, else tier-3 PARAM\_LIMITS (raw → × display\_mult for the spinbox).
- .af values outside the FC tier-1 ceiling are flagged (orange) but still blocked by the FC clamp at write time.
- Migration: every shipped .af gets a [LIMITS] block generated from its current values (bracket ×2, round). The tier-3 fallback exists only so a new/unknown frame is still editable.

## 6. 128-param → class mapping (provisional)

Legend: C = shared class (ceiling via section 4.1), X = explicit per-param (lo, hi), U = unused (kept as X (0,255)), R = read-only (BootDiag / reset cause — accepted, not clamped meaningfully).

### Gains — attitude rate

| Tag | Param         | Type | Class          | Scale    | Raw bound |
|-----|---------------|------|----------------|----------|-----------|
| 0   | RollRateKp    | F    | classGainRateP | 0.005    | 0 – 3.0   |
| 5   | PitchRateKp   | F    | classGainRateP | 0.005    | 0 – 3.0   |
| 10  | YawRateKp     | F    | classGainRateP | 0.005    | 0 – 3.0   |
| 11  | RollRateKd    | F    | classGainRateD | 0.0001   | 0 – 0.2   |
| 27  | PitchRateKd   | F    | classGainRateD | 0.0001   | 0 – 0.2   |
| 90  | YawRateKd     | F    | classGainRateD | 0.000025 | 0 – 0.05  |
| 108 | YawRateKi     | F    | classGainRateI | 0.001    | 0 – 0.2   |
| 109 | YawRateIntLim | F    | X (0, 2.0)     | 0.01     | 0 – 2.0   |

### Gains — attitude angle

| Tag | Param               | Type | Class           | Scale    | Raw bound |
|-----|---------------------|------|-----------------|----------|-----------|
| 2   | RollAngleQKp        | F    | classGainAngleQ | 1.0      | 0 – 20    |
| 7   | PitchAngleQKp       | F    | classGainAngleQ | 1.0      | 0 – 20    |
| 96  | YawAngleQKp         | F    | classGainAngleQ | 1.0      | 0 – 20    |
| 23  | RollAngleQKi        | F    | classGainAngleI | 0.05     | 0 – 25    |
| 24  | PitchAngleQKi       | F    | classGainAngleI | 0.05     | 0 – 25    |
| 97  | YawAngleQKi         | F    | classGainAngleI | 0.05     | 0 – 25    |
| 4   | RollAngleQIntLimit  | F    | classRateIntLim | 0.015    | 0 – 30    |
| 9   | PitchAngleQIntLimit | F    | classRateIntLim | 0.015    | 0 – 30    |
| 98  | YawAngleQIntLimit   | F    | classRateIntLim | 0.000873 | 0 – 30    |

### Altitude & nav gains

| Tag | Param           | Type | Class        | Scale   | Raw bound |
|-----|-----------------|------|--------------|---------|-----------|
| 1   | AltPosKi        | F    | classGainAlt | 0.00046 | 0 – 3.66  |
| 6   | AltPosKp        | F    | classGainAlt | 0.0183  | 0 – 3.66  |
| 29  | AltVelKd        | F    | classGainAlt | 1.0     | 0 – 3.66  |
| 99  | AltPosIntLimit  | F    | classGainAlt | 0.05    | 0 – 3.66  |
| 101 | AltVelKi        | F    | classGainAlt | 0.00027 | 0 – 3.66  |
| 120 | AltROCKp        | F    | classGainAlt | 0.001   | 0 – 3.66  |
| 28  | NavVelKp        | F    | classGainNav | 0.06    | 0 – 15    |
| 40  | NavPosIntLimit  | F    | classGainNav | 1.0     | 0 – 15    |
| 48  | NavCrossTrackKp | F    | classGainNav | 0.01    | 0 – 15    |
| 56  | NavPosKp        | F    | classGainNav | 0.0165  | 0 – 15    |
| 60  | NavPosKi        | F    | classGainNav | 0.004   | 0 – 15    |

### Misc gains

| Tag | Param         | Type | Class        | Scale | Raw bound |
|-----|---------------|------|--------------|-------|-----------|
| 18  | RollCamKp     | F    | classGainCam | 0.1   | 0 - 20    |
| 25  | PitchCamKp    | F    | classGainCam | 0.1   | 0 - 20    |
| 72  | KFAccUBiasVar | F    | classGainKf  | 1.0   | 0 - 20    |
| 110 | KFBaroVar     | F    | classGainKf  | 1.0   | 0 - 20    |
| 111 | KFAccUVar     | F    | classGainKf  | 1.0   | 0 - 20    |
| 31  | MadgwickKpMag | F    | X (0, 0.55)  | 0.01  | 0 - 0.55  |
| 38  | MadgwickKpAcc | F    | X (0, 0.5)   | 1.0   | 0 - 0.5   |
| 30  | Horizon       | F    | X (0, 50)    | 1.0   | 0 - 50    |
| 52  | AccConfSD     | F    | X (0, 50)    | 1.0   | 0 - 50    |

### Angles & rates (deg / deg/s)

| Tag | Param                   | Type | Class             | Scale   | Raw bound      |
|-----|-------------------------|------|-------------------|---------|----------------|
| 33  | NavMagVar               | F    | classAngle        | DEG_RAD | 0 - 1.047      |
| 67  | FWMaxClimbAngle         | F    | classAngle        | DEG_RAD | 0 - 1.047      |
| 68  | NavMaxAngle             | F    | classAngle        | DEG_RAD | 0 - 1.047      |
| 74  | MaxPitchAngle           | F    | classAngle        | DEG_RAD | 0 - 1.047      |
| 76  | MaxRollAngle            | F    | classAngle        | DEG_RAD | 0 - 1.047      |
| 78  | NavHeadingTurnRate      | F    | classAngle        | DEG_RAD | 0 - 1.047      |
| 113 | FWRollControlPitchLimit | F    | X (0.785, 1.047)  | DEG_RAD | 45-60 deg      |
| 81  | FWBoardPitchAngle       | F    | classAngleBipolar | DEG_RAD | -0.349 - 0.349 |
| 63  | MaxYawRate              | F    | classRate         | DEG_RAD | 0 - 10.472     |
| 82  | MaxRollRate             | F    | classRate         | DEG_RAD | 0 - 10.472     |
| 83  | MaxPitchRate            | F    | classRate         | DEG_RAD | 0 - 10.472     |
| 88  | MaxHeadingRate          | F    | classRate         | DEG_RAD | 0 - 10.472     |

### Percent / fraction

| Tag | Param                   | Type | Class         | Scale | Raw bound  |
|-----|-------------------------|------|---------------|-------|------------|
| 19  | EstCruiseThr            | F    | classPct      | 0.01  | 0 - 1.0    |
| 20  | StickHysteresis         | F    | classPct      | 0.01  | 0 - 1.0    |
| 21  | FWClimbThrottle         | F    | classPct      | 0.01  | 0 - 1.0    |
| 22  | PercentIdleThr          | F    | classPct      | 0.01  | 0 - 1.0    |
| 39  | RollCamTrim             | F    | classPct      | 0.01  | 0 - 1.0    |
| 58  | Balance                 | F    | X (-0.2, 0.5) | 0.01  | -0.2 - 0.5 |
| 64  | FWRollPitchFF           | F    | classPct      | 1.0   | 0 - 1.0    |
| 65  | FWPitchThrottleFF       | F    | classPct      | 0.01  | 0 - 1.0    |
| 70  | FWAileronDifferential   | F    | classPct      | 0.01  | 0 - 1.0    |
| 80  | YawSymmetryFactor       | F    | classPct      | 0.01  | 0 - 1.0    |
| 86  | FWAileronRudderMix      | F    | classPct      | 0.01  | 0 - 1.0    |
| 87  | FWAltSpoilerFF          | F    | classPct      | 0.01  | 0 - 1.0    |
| 102 | AltThrottleCompLimit    | F    | X (0, 0.25)   | 0.01  | 0 - 0.25   |
| 112 | FWStickScale            | F    | X (0.15, 1.0) | 0.01  | 0.15 - 1.0 |
| 114 | AHThrottleMovingWindow  | F    | classPct      | 0.01  | 0 - 1.0    |
| 119 | BatteryAlarmPct         | F    | X (0.05, 0.5) | 0.01  | 0.05 - 0.5 |
| 122 | RudderMotorFF           | F    | X (0, 20)     | 0.01  | 0 - 20     |
| 69  | FWSpoilerDecayPercentPS | F    | classPctS     | 0.001 | 0 - 0.5    |

| Tag | Param                        | Type | Class      | Scale | Raw bound |
|-----|------------------------------|------|------------|-------|-----------|
| 79  | AltHoldThrCompDecayPercentPS | F    | X (0, 0.2) | 1.0   | 0 - 0.2   |

### Scalar (Hz / V / A / m / m/s / mAh / s)

| Tag | Param              | Type | Class       | Scale | Raw bound    |
|-----|--------------------|------|-------------|-------|--------------|
| 26  | ServoLPFHz         | F    | classHz     | 1.0   | 0 - 255      |
| 77  | YawLPFHz           | F    | X (25, 255) | 1.0   | 25 - 255     |
| 17  | LowVoltThres       | F    | X (9, 20)   | 1.0   | 9 - 20       |
| 85  | VoltScale          | F    | classVolts  | 1.0   | 0 - 255      |
| 84  | CurrentScale       | F    | classAmps   | 1.0   | 0 - 255      |
| 32  | NavRTHAlt          | F    | X (0, 30)   | 1.0   | 0 - 30       |
| 106 | NavProxAltM        | F    | X (0, 10)   | 1.0   | 0 - 10       |
| 107 | NavProxRadiusM     | F    | X (0, 30)   | 1.0   | 0 - 30       |
| 115 | NavFenceRadiusM    | F    | X (0, 1000) | 1.0   | 0 - 1000     |
| 116 | DiveRecoverAlt     | F    | X (10, 200) | 1.0   | 10 - 200     |
| 45  | MaxDescentRateDmpS | F    | X (0, 50)   | 1.0   | 0 - 50       |
| 103 | VRSROC             | F    | X (-50, 0)  | 1.0   | -50 - 0      |
| 105 | AHROCWindowMPS     | F    | X (0, 30)   | 1.0   | 0 - 30       |
| 53  | BatteryCapacity    | F    | classMah    | 1.0   | 1500 - 10000 |
| 46  | DescentDelayS      | U8   | X (0, 30)   | 1     | 0 - 30       |
| 118 | FailsafeDelay      | U8   | X (0, 60)   | 1     | 0 - 60       |

### Enums / bitmasks (explicit = enum range)

| Tag | Param            | Type | Bound                 |
|-----|------------------|------|-----------------------|
| 3   | ArmingMode       | U8   | 0 - 1                 |
| 8   | RFSensorType     | U8   | 0 - MaxSonarcm        |
| 12  | IMUFiltType      | U8   | 0 - IMU_FILT_TYPE_MAX |
| 13  | BBLogType        | U8   | 0 - logYaw            |
| 14  | RxType           | U8   | 0 - UnknownRx         |
| 15  | Config1Bits      | U8   | 0 - CONFIG_BITS_MAX   |
| 35  | ESCType          | U8   | 0 - MotorsOff         |
| 36  | RCChannels       | U8   | 0 - 16                |
| 43  | AFTType          | U8   | 0 - AFUnknown         |
| 44  | TelemetryType    | U8   | 0 - u8Telemetry       |
| 47  | GyroLPFSel       | U8   | 0 - GYRO_LPF_SEL_MAX  |
| 51  | ServoSense       | U8   | 0 - 127               |
| 71  | ASSensorType     | U8   | 0 - noAS              |
| 73  | Config2Bits      | U8   | 0 - CONFIG_BITS_MAX   |
| 89  | AccLPFSel        | U8   | 0 - ACC_LPF_SEL_MAX   |
| 92  | ThrottleGainRate | U8   | 0 - 100               |
| 100 | MotorStopSel     | U8   | 0 - landContactSw     |
| 117 | FailsafeAction   | U8   | 0 - 2                 |

### Channel maps (U8, 0 - 15)

16, 37, 41, 42, 49, 50, 54, 55, 59, 93, 94, 95 — Map[ . . . ] RC channel selectors. Explicit bound 0 - 15.

## Read-only / diagnostics (accepted, clamped to 0-255)

104 BootDiag, 127 PowerResetCause — R.

## Unused slots

34, 57, 61, 62, 66, 75, 91, 121, 123, 124, 125, 126 — U (0, 255).

## 7. Migration plan

1. **FC:** add `cls` field to `ParamMetaEntry`; add `ParamClassBounds[]`; re-map all 128 `FL0AT()/U8()` macro calls to class ids (section 6); implement the clamp in `ProcessParamsWrite + PackParametersToConfig`. → `ParamTableCRC` changes, layout identical → **cal preserved**, no magic bump.
2. **GCS:** add `ParamClassBounds` table; derive `PARAM_LIMITS`; add `.af [LIMITS]` parser + slider wiring; add unit test.
3. **Revert the 12-tag limit hacks** (Aug 2026) back to clean class ceilings; the per-frame precision moves into each `.af [LIMITS]` block.
4. **.af migration:** generate `[LIMITS]` blocks for all shipped frames (bracket current values  $\times 2$ ), review, ship.
5. **Validate:** tuner asserts tuned values  $\leq$  FC ceiling for all 17 frames; GCS `py_compile` + unit tests.

## 8. Bounds-tuning feedback loop

Class ceilings are a first guess, not gospel. Expect to tighten/loosen them as real tuned ranges are learned. The GCS readback verification already provides the signals — no new GCS plumbing needed:

- **Too tight:** `_verify_parameters` (`parameter_window.py`) reports `Written=X` → Clamped to `Y` (`max=Z`) on the bulk-readback check (tag 71) when the FC clamped a write to the ceiling. Widen the class (or promote the param to a one-off with a larger bound).
- **Too loose:** a param's tuned values across all frames cluster well below the ceiling — the class can be tightened for better fallback-slider resolution.

`[LIMITS]` blocks per airframe should be tracked so that tuned-value distributions can be reviewed against the class table in one place.

## 9. Open questions

1. `classGainAlt` (0 – 3.66) groups gains spanning 4 orders of scale (`AltVelKi` 0.00027 → `AltVelKd` 1.0). The shared raw ceiling keeps the FC clamp simple, but the tier-3 fallback slider is coarse for small-scale members — acceptable since real frames carry `[LIMITS]`, but confirm we want this grouping vs. separate one-off ceilings per alt gain.
2. `MadgwickKpMag` (scale 0.01) and `MadgwickKpAcc` (scale 1.0) are treated as one-offs — confirm.
3. Exact class ceiling numbers (section 4.1) need validation against all `.af` files during implementation.